RetroBASIC ユーザーズマニュアル

Version 1.35 / PC-8801 style BASIC environment

RetroBASIC Version 1.35

RetroBASICは、1983年頃のN88-BASIC、MSX BASIC、GW-BASICなどの雰囲気を、 現在のMacで気軽に楽しむための行番号方式BASICインタープリタです。

厳密な特定機種のエミュレーターではありません。当時よく使われていた命令、表示形式、 操作感を再現しつつ、macOS上で扱いやすいテキスト環境として実装されています。

目次

1. 動作環境と起動
2. 最初のプログラム
3. プログラムの入力と編集
4. キーボード操作
5. 画面設定
6. BASICの基礎
7. コマンドリファレンス
8. ステートメントリファレンス
9. 関数リファレンス
10. ファイルの保存と読み込み
11. 実行速度
12. エラーと対処
13. サンプルプログラム
14. 制限事項

1. 動作環境と起動

1.1 動作環境

- macOS 13以降
- 同梱アプリはApple Silicon Mac向け
- GUI版のRetroBASIC.appと、ターミナル用のretrobasic-cliを収録

1.2 GUI版の起動

FinderでRetroBASIC.appをダブルクリックします。起動すると、次のようなメッセージと Okプロンプトが表示されます。

```
RetroBASIC Version 1.35
80's BASIC style interpreter, circa 1983
20289 Bytes free
Ok
```

起動時には、BASIC命令を入力しやすいよう、macOSの入力ソースが英数入力へ一度だけ 切り替わります。起動後は通常のmacOS操作で日本語IMEへ戻せます。

1.3 CLI版の起動

ターミナルでRetroBasicフォルダへ移動し、次のように実行します。

```
./retrobasic-cli
```

BASICファイルを指定すると、読み込んですぐに実行します。

```
./retrobasic-cli samples/GUESS.bas
```

実行後も対話モードを続ける場合は-iを加えます。

```
./retrobasic-cli samples/GUESS.bas -i
```

CLI版は同じインタープリタを使用しますが、スクリーンエディタと非同期キー入力を利用する ゲームはGUI版向けです。

2. 最初のプログラム

Okの下で、次の3行を入力します。各行の最後でReturnを押してください。

```
10 PRINT "HELLO, RETROBASIC!"
20 PRINT "2 + 3 =";2+3
30 END
```

行番号付きの入力は、すぐには実行されず、プログラムとして記憶されます。入力した内容を 確認するにはLIST、実行するにはRUNを入力します。

```
LIST
RUN
```

実行結果は次のようになります。

```
HELLO, RETROBASIC!
2 + 3 = 5
Ok
```

2.1 イミディエイトモード

行番号を付けない入力は、その場ですぐ実行されます。これをイミディエイトモードと呼びます。

```
PRINT SQR(144)
```

計算の確認、変数の変更、画面設定、ファイル操作などに利用できます。

3. プログラムの入力と編集

3.1 行番号

プログラム行は、行番号の小さい順に保存されます。行番号には0〜65529を使用できます。 通常は、後から行を追加できるよう10、20、30のように間隔を空けます。

```
10 PRINT "FIRST"
30 PRINT "THIRD"
```

後から20行を入力すると、10行と30行の間へ入ります。

```
20 PRINT "SECOND"
```

3.2 行の置き換えと削除

既存行と同じ行番号を入力すると、その行全体を置き換えます。

```
20 PRINT "NEW SECOND LINE"
```

行番号だけを入力すると、その行を削除します。

```
20
```

3.3 1行に複数の命令を書く

コロン:で区切ると、1つの行へ複数の命令を書けます。

```
10 CLS:COLOR 6,0:PRINT "READY":BEEP
```

3.4 コメント

REMまたはアポストロフィ'以降はコメントです。

```
10 REM START OF PROGRAM
20 A=A+1 ' COUNT UP
```

3.5 大文字と小文字

文字列の外で入力した小文字は自動的に大文字へ変換されます。

```
10 print "Hello"
```

この行はLISTすると次のように表示されます。

```
10 PRINT "Hello"
```

3.6 スクリーンエディタ

GUI版では、LISTなどで画面に表示されたプログラム行を直接修正できます。

1. LISTでプログラムを表示します。
2. 矢印キーで修正する行へカーソルを移動します。
3. 文字を修正します。
4. Returnを押すと、画面上の論理行全体が再入力されます。

画面幅を超えて折り返した行も、Returnを押すと1つのBASIC行として復元されます。

通常は上書きモードです。Control+IまたはEditメニューの「Insert Mode」で挿入モードを 切り替えられます。

- ブロックカーソル：上書きモード
- 細い縦カーソル：挿入モード
- 上書きモードのBackspace / Delete：文字を空白へ置き換える
- 挿入モードのBackspace / Delete：文字を削除して後続文字を左へ詰める

矢印キーが編集操作になるのは入力待ち中だけです。プログラム実行中はINKEY$用の キーとして扱われます。

4. キーボード操作

操作	キーまたはメニュー
プログラムの中断	Control+CまたはCommand+.
挿入 / 上書き切り替え	Control+IまたはEdit → Insert Mode
カナ / GRPH文字パレット	Command+KまたはInput → Character Palette…
画面内のカーソル移動	矢印キー
直前の文字を削除	Backspace
カーソル位置を削除	Delete
クリップボードから貼り付け	Command+V
画面全体をコピー	Command+C
画面を拡大する	Command++
画面を縮小する	Command+-
CRT風表示を切り替える	View → CRT Effect
アプリを終了	Command+Q

複数行のBASICリストを貼り付けると、各行が順番に入力されます。雑誌やWebページから プログラムを転記するときに便利です。

Input → Character Palette…を選ぶと、半角カタカナとGRPH文字の入力パレットが開きます。 Katakana / GRPHを切り替え、表示された8×8文字をクリックすると、パレット下部の入力欄へ 文字が追加されます。←ボタンで末尾の1文字を削除でき、GOボタンを押すと入力欄の全文字が 現在のBASICカーソル位置へまとめて挿入され、ターミナルへフォーカスが戻ります。カナの 濁点･ 半濁点は独立した1文字で、ｶﾞはｶとﾞの2文字を使います。

5. 画面設定

5.1 画面の消去

```
CLS
CLS 1
CLS 2
CLS 3
```

引数なしまたはCLS 3はテキスト画面とグラフィック画面の両方を消去します。 CLS 1はテキスト画面だけ、CLS 2はグラフィック画面だけを消去します。

5.2 画面幅

画面は80×25文字または40×25文字です。

```
WIDTH 80
WIDTH 40
```

WIDTHを実行するとテキスト画面とグラフィック画面が消去されます。40桁モードでは ウィンドウの大きさを変えず、文字を横方向に2倍で表示します。グラフィック解像度も連動し、 WIDTH 40では320×200、WIDTH 80では640×200になります。起動時は80桁です。

40と80以外を指定するとIllegal function callになります。

5.3 文字と画面のドット

文字は8×8ドットのキャラクタで表示されます。80桁×25行はちょうど640×200ドットで、 グラフィック画面と同じドット空間です。そのためテキストと図形はドット単位でぴったり 重なります。たとえばLINE (0,64)-(639,64)はちょうど9行目の文字の上端に並びます。

字形は1983年頃の国産8ビット機（N88-BASIC / PC-8801系）の雰囲気に合わせた ドットフォントです。英数記号に加え、PC-8001/8801系の文字パターンに合わせた半角カタカナ（ｱｰﾝ、濁点･ 半濁点、｢｣､･ｰ）と56個のGRPH文字を内蔵しています。

```
PRINT "ｺﾝﾆﾁﾊ ｾｶｲ"
```

GRPH文字には、段階的な塗りつぶし、罫線、三角形、トランプ記号、丸、斜線、 円･ 年･ 月･ 日･ 時･ 分･ 秒があります。文字コードは80〜9FおよびE0〜F7（16進）です。 これらは固定の文字ROM字形であり、PC-8001式の2×4セミグラフィックには対応していません。

ひらがなや漢字は当時の8ビット機と同様、文字ROMに相当するものを持ちません。入力しても 消えることはありませんが、8×8ドットへ機械的に縮小した粗い表示になります。日本語を きれいに出したい場合は半角カタカナを使ってください。

画面は実機と同じく1ドットが横1×縦2の比率で表示されます。表示倍率は整数倍のみで、 Command++ / Command+-（View → Zoom In / Zoom Out）で切り替えます。等倍で 640×400ピクセル、2倍で1280×800ピクセルです。起動時は画面に収まる最大の倍率が 選ばれます。整数倍に固定しているのは、ドットが均一な大きさで並ぶようにするためです。

View → CRT Effectを有効にすると、柔らかな発光、走査線、ごく弱い周辺減光を加えたCRT風表示に 切り替わります。効果は静的で、文字やグラフィックの元データ、画面の整数倍表示には影響 しません。設定は次回起動時にも引き継がれます。初期状態はオフです。

5.4 文字色と背景色

```
COLOR 前景色,背景色
```

色番号は次のとおりです。

番号	色
0	黒
1	青
2	赤
3	紫
4	緑
5	水色
6	黄
7	白

起動時は白文字、黒背景です。

```
COLOR 7,1
PRINT "WHITE ON BLUE"
COLOR 0,7
PRINT "BLACK ON WHITE"
```

背景色を省略すると、現在の背景色を維持して文字色だけを変更します。

```
COLOR 6
```

5.5 カーソル位置

```
LOCATE X,Y
```

Xは列、Yは行で、どちらも0から数えます。80桁ではXが0〜79、40桁では0〜39、 Yは0〜24です。範囲外の値は画面内へ丸められます。

3番目の引数も受理しますが、カーソル表示指定としては現在使用されません。

```
LOCATE 10,5
PRINT "HERE"
```

5.6 音

```
BEEP
```

MSX1の標準BEEPを再現した短い矩形波を、RetroBASICのPSG音源で鳴らします。 チャンネルAのE6相当（約1316Hz）、音量7、長さ40msです。バックグラウンドで PLAYしている曲がある場合は停止し、BEEPを優先します。

```
PLAY MML文字列1[,MML文字列2[,MML文字列3]]
```

AY-3-8910を想定した3チャンネルPSG音源でMMLを演奏します。1番目から順に チャンネルA、B、Cへ割り当てられ、各文字列は文字列式や文字列変数でも指定できます。

```
A$="T120O4L8V13 CDEFGAB>C"
B$="T120O3L4V9 CEGC"
PLAY A$,B$
```

既定のMF（フォアグラウンド）では、PLAYは曲が終わるまでBASICプログラムの実行を 待ちます。演奏中もControl+CまたはCommand+.で中断できます。

```
PLAY "MFT120O4L8 CDEFGAB>C"
PRINT "FINISHED"
```

MB（バックグラウンド）を指定すると、曲を演奏しながら直後のステートメントへ進みます。

```
PLAY "MBT120O4L8 CDEFGAB>C"
PRINT "MUSIC IS PLAYING"
```

MF / MBは次回以降のPLAYにも引き継がれます。新しいPLAYを実行すると、それまで 演奏していた曲を停止して新しい曲へ置き換えます。バックグラウンド演奏だけを停止するには 空のMMLを指定します。

```
PLAY ""
```

バックグラウンド演奏はPLAYから既に戻っているため、Control+CではなくPLAY ""で 停止します。音符に関する各MML文字列の初期値はT120 O4 L4 V8 Q8 @0で、 チャンネルごとに独立しています。

MML	説明
C〜B	ドーシ。直後に音長、付点を指定できます（例: C8.）。
+ / # / -	直前の音名を半音上げ / 半音下げします。
R	休符。音符と同じ音長、付点を指定できます。
T32〜T255	テンポ（1分あたりの四分音符数）を指定します。
O1〜O8	オクターブを指定します。>で1つ上げ、<で1つ下げます。
L1〜L64	省略時の音長を指定します。L4は四分音符です。
V0〜V15	4ビット相当の音量を指定します。0は無音です。
Q1〜Q8	音長のうち実際に鳴らす割合を8分率で指定します。
N0〜N96	音番号で指定します。N0は休符、N1はC1です。
.	付点。複数指定すると1/2、1/4…の長さを順に加えます。
&	前後の音を切れ目なくつなぎます。
MF	フォアグラウンド演奏。曲が終わるまでPLAYで待ちます（初期値）。
MB	バックグラウンド演奏。演奏しながら後続のBASIC文を実行します。
@0 / @1 / @2	トーン / ノイズ / トーン + ノイズを選びます（RetroBASIC拡張）。

トーンはAYの1.789773MHzクロックと12ビット周期に合わせて音程を量子化し、3チャンネルを モノラルでミックスします。ノイズジェネレーターは3チャンネルで共有されます。

5.7 グラフィック画面

GUI版は、初代PC-8801風の8色グラフィック画面をテキスト画面の背面に重ねて表示します。 解像度は文字幅と連動し、WIDTH 40では320×200、WIDTH 80では640×200です。 左上は常に(0,0)で、右下は40桁時が(319,199)、80桁時が(639,199)です。

```
WIDTH 80
PSET (100,50),2
PRESET (100,50)
LINE (20,20)-(300,150),5
LINE (40,40)-(200,100),6,B
LINE (220,40)-(380,100),3,BF
CIRCLE (320,100),80,7
PAINT (320,100),1,7
```

LINE ... ,BFの色、またはPAINTの塗色には、数値の代わりにSHARP X1 Hu-BASIC形式の タイル文字列を指定できます。タイルは横8ドット固定で、1行ごとに青· 赤· 緑の3バイトを 並べます。各バイトの最上位ビットが左端です。3プレーンの同じ位置のビットを重ねることで、 色番号0〜7を作ります。タイルは画面左上(0,0)を基準に繰り返されるため、別々に塗った 隣接領域でも模様が連続します。

```
' 1行の色は左から 紫、赤、水色、緑 の繰り返し
T$=HEXCHR$("AA CC 33")
LINE (20,20)-(180,80),T$,BF

' 3行を青、赤、緑で繰り返す
T$=HEXCHR$("FF0000 00FF00 0000FF")
LINE (220,20)-(380,80),7,B
PAINT (230,30),T$,7
```

タイル文字列の長さは3の倍数（1〜255バイト）でなければなりません。PAINTでは境界色を 複数指定でき、PAINT (X,Y),T$,7,6のようにカンマで続けます。単色の従来構文もそのまま 利用できます。

STEP (X,Y)を使うと、直前のグラフィック座標からの相対位置を指定できます。

```
PSET (100,100),7
LINE -(200,100),7
LINE -STEP (0,50),7
```

WIDTHを切り替えるとグラフィック面も新しい解像度で作り直され、描画内容は消去されます。 CIRCLEの縦横比は現在の解像度に合わせて自動調整されます。SCREEN命令、640×400 モノクロモード、VRAMページ切り替え、WINDOWによる座標変換には対応していません。 CLI版でも命令とPOINTの計算は行いますが、グラフィック画面そのものは表示しません。

6. BASICの基礎

6.1 変数

変数名は英字で始めます。値を代入していない変数は、数値なら0、文字列なら空文字列です。

```
A=10
NAME$="RETROBASIC"
LET COUNT%=3
```

LETは省略できます。

サフィックス	種類	例
なし、!、#	数値	A、X#
%	16ビット整数	COUNT%
$	文字列	NAME$

内部の数値計算は倍精度浮動小数点で行われます。%変数へ代入すると整数へ丸められ、 範囲外ではOverflowになります。

6.2 数値定数

```
123
-3.5
.25
1.2E3
&HFF
&O377
```

&Hは16進数、&Oまたは先頭の&は8進数です。

6.3 文字列

文字列は二重引用符で囲み、+で連結できます。

```
A$="RETRO"
B$="BASIC"
PRINT A$+B$
```

6.4 配列

```
DIM A(20)
DIM NAME$(10,5)
```

DIMの数値は各次元の最大添字です。DIM A(20)ではA(0)からA(20)まで使えます。

未宣言の配列を使うと、各次元の添字10まで自動確保されます。一度宣言した配列を再度 DIMするとDuplicate Definitionになります。ERASEで配列を破棄できます。

6.5 演算子

主な優先順位は、強いものから概ね次の順です。

1. 括弧、関数、変数、定数
2. べき乗 ^
3. 単項符号 + -
4. 乗除 * /

5. 整数除算 \
6. 剰余 MOD
7. 加減算・文字列連結 + -
8. 比較 = <> < > <= >=
9. NOT
10. AND
11. OR
12. XOR

比較が真なら-1、偽なら0になります。論理演算はMS BASICと同様に整数のビット演算です。

```
IF A>=10 AND A<20 THEN PRINT "IN RANGE"
```

6.6 PRINTの書式

```
PRINT "VALUE=";A
PRINT A,B,C
PRINT TAB(20);"TITLE"
PRINT SPC(5);"INDENT"
PRINT USING "###.##";12.345
? "QUESTION MARK IS PRINT"
```

- セミコロン; : 項目を続けて表示
- カンマ, : 次の14桁ゾーンへ移動
- TAB(n) : n桁目へ移動。現在位置を過ぎていれば改行
- SPC(n) : n個の空白を表示
- 行末の;または, : 改行を抑制
- ? : PRINTの省略形

正の数には先頭空白と末尾空白が付きます。小数は.5や-.25のように表示されます。

PRINT USINGでは、書式文字列に従って数値や文字列を整形できます。

```
PRINT USING "###.##";12.345
PRINT USING "#,###.##";1234.5
PRINT USING "+##.##";-3.5
PRINT USING "NAME=\       \";"RETROBASIC"
F$="**$###.##":PRINT USING F$;42.5
```

書式	意味
#	数字位置。値を右寄せし、指定した小数桁へ丸めます。
.	小数点位置。
,	小数点より左に置くと3桁区切りを表示します。
+	先頭または末尾の符号を常に表示します。
末尾の-	負数のとき末尾にマイナスを表示します。
**	先頭の空白を*で埋めます。
$$ / **$	$を数字の直前に表示します。**$は*埋めも行います。
^^^^	指数形式で表示します。
!	文字列の先頭1文字を表示します。
&	文字列全体を表示します。
\ \	2本の\を含む幅に文字列を切り詰め、左寄せします。
_	次の書式文字を通常の文字として表示します。

値が数値フィールドに収まらない場合は、先頭に%が付きます。値の数が書式内の フィールド数より多い場合は、書式文字列を先頭から繰り返します。通常は最後に改行し、 式リストの末尾を;または,にすると通常のPRINTと同様に次の出力位置を制御できます。

6.7 INPUT

```
INPUT A
INPUT "AGE";A
INPUT "X,Y";X,Y
LINE INPUT "NAME: ";N$
```

数値入力が不正な場合や項目数が合わない場合は?Redo from startと表示され、再入力に なります。文字列入力には引用符を付けなくても構いません。

LINE INPUTはカンマを区切りとして扱わず、1行全体を文字列へ代入します。

7. コマンドリファレンス

7.1 プログラム操作

コマンド	構文	説明
RUN	RUN	先頭行から実行します。変数と実行時状態を初期化します。
RUN	RUN 行番号	指定行から実行します。
RUN	RUN "ファイル名"	ファイルを読み込んで実行します。
LIST	LIST	プログラム全体を表示します。
LIST	LIST 100	指定行だけを表示します。
LIST	LIST 100-200	指定範囲を表示します。
LIST	LIST -200	先頭から200行まで表示します。
LIST	LIST 100-	100行から最後まで表示します。
RENUM	RENUM	全行を10から10刻みで再番号付けします。
RENUM	RENUM 新開始行[,旧開始行[,増分]]	指定範囲の行番号をまとめて変更します。
AUTO	AUTO [開始行][,増分]	入力行へ行番号を自動的に付けます。
NEW	NEW	プログラムと変数を消去します。
CLEAR	CLEAR	プログラムを残し、変数・配列・スタックを初期化します。
CONT	CONT	STOPまたは中断した位置から再開します。

プログラムを編集した後や、再開位置が失われた後のCONTはCan't continueになります。

RENUMの既定値は、新開始行10、旧開始行0、増分10です。引数を省略する場合はカンマを 残せます。

```
RENUM
RENUM 100
RENUM 1000,500,20
RENUM 100,,5
```

行番号本体に加えて、GOTO、GOSUB、THEN、ELSE、ON GOTO、ON GOSUB、 RESTORE、RUNが参照する行番号も更新されます。文字列、コメント、DATA内の数値は 変更されません。変更後の番号が既存行と衝突する場合や65529を超える場合は、プログラムを 変更せずIllegal function callを表示します。RENUMはイミディエイトモードで使用します。

AUTOの開始行と増分の既定値は、どちらも10です。開始行を省略して増分だけを指定する 場合は、AUTO ,5のように入力します。開始行の後をカンマで終えると、直前のAUTOで 指定した増分を再利用します。表示された番号に続けてプログラムを入力すると、 Enterのたびに次の番号が表示されます。既存行の番号には*が付きます。そこで空のまま Enterを押すと既存行を変更せず次へ進みます。終了するにはCtrl+Cを押します。

7.2 ファイル操作

コマンド	構文	説明
SAVE	SAVE "名前"	現在のプログラムを保存します。
LOAD	LOAD "名前"	プログラムを読み込み、現在の内容と置き換えます。
MERGE	MERGE "名前"	ファイルの行を現在のプログラムへ統合します。
FILES	FILES	保存フォルダの.basファイルを一覧表示します。
KILL	KILL "名前"	指定ファイルを削除します。

詳細は「10. ファイルの保存と読み込み」を参照してください。

7.3 デバッグ

コマンド	説明
TRON	実行する行番号を[100]の形式で表示します。
TROFF	行番号トレースを終了します。

8. ステートメントリファレンス

8.1 代入・入出力

ステートメント	構文例	説明
LET	LET A=10	変数または配列要素へ代入します。LETは省略可能です。
PRINT	PRINT A;B	値を表示します。?も使用できます。
PRINT USING	PRINT USING "###.##";A	書式文字列に従って値を表示します。
LPRINT	LPRINT A	現在はPRINTと同じ画面へ表示します。
INPUT	INPUT "AGE";A	キーボードから値を入力します。
LINE INPUT	LINE INPUT A$	1行全体を文字列として入力します。

8.2 条件分岐

```
IF 条件 THEN ステートメント
IF 条件 THEN 行番号
IF 条件 GOTO 行番号
IF 条件 THEN ステートメント ELSE ステートメント
```

例：

```
100 IF A=0 THEN PRINT "ZERO" ELSE PRINT "NON-ZERO"
110 IF A<0 THEN 500
```

同じ行に入れ子のIFを書くこともできます。

8.3 GOTO・GOSUB

```
GOTO 100
GOSUB 1000
RETURN
```

GOSUBは戻り位置を記憶して指定行へ移動し、RETURNで呼び出し元へ戻ります。

```
10 GOSUB 1000
20 PRINT "BACK"
30 END
1000 PRINT "SUBROUTINE"
1010 RETURN
```

8.4 ON GOTO・ON GOSUB

```
ON N GOTO 100,200,300
ON N GOSUB 1000,2000,3000
```

Nが1なら1番目、2なら2番目の行番号を選びます。Nが0、または一覧の数より大きい場合は 何もせず次へ進みます。Nは0〜255の範囲で指定します。

8.5 FOR・NEXT

```
FOR I=1 TO 10
  PRINT I
NEXT I
```

増分を変えるにはSTEPを使います。

```
FOR I=10 TO 0 STEP -2
  PRINT I
NEXT I
```

複数のループをNEXT J,Iのように閉じることもできます。当時のBASICと同様、開始値が 終了値を既に越えていても、ループ本体を最低1回実行してからNEXTで判定します。

8.6 WHILE・WEND

```
WHILE A<10
  A=A+1
  PRINT A
WEND
```

条件が0以外の間、処理を繰り返します。入れ子にも対応しています。

8.7 DATA・READ・RESTORE

```
10 READ A$,B,C
20 PRINT A$,B,C
30 DATA APPLE,10,20
```

文字列のDATAには引用符を使用できます。

```
100 DATA "HELLO, WORLD",42
```

RESTOREは読み込み位置を先頭へ戻します。行番号を指定すると、その行以降の最初の DATAへ移動します。

```
RESTORE
RESTORE 1000
```

8.8 DIM・ERASE

```
DIM A(100),B$(20,5)
ERASE A,B$
```

ERASE後は同じ配列名を再度DIMできます。

8.9 DEF FN

1行形式のユーザー定義関数を作成します。

```
10 DEF FNSQ(X)=X*X
20 PRINT FNSQ(12)
```

複数の引数も使用できます。

```
10 DEF FNAVG(A,B)=(A+B)/2
```

8.10 RANDOMIZE

```
RANDOMIZE
RANDOMIZE 1234
```

引数なしでは現在時刻、引数ありでは指定値を乱数系列の種にします。

8.11 SWAP

```
SWAP A,B
SWAP A$,B$
```

2つの変数または配列要素の値を交換します。数値と文字列を混在させることはできません。

8.12 POKE

```
POKE アドレス,値
```

0〜65535の仮想メモリへ、0〜255の値を書き込みます。PEEK関数で読み出せます。 実機のハードウェアへは影響しません。

8.13 終了と停止

ステートメント	説明
END	プログラムを終了します。
STOP	Break in 行番号を表示して停止します。CONTで再開できます。

8.14 その他

ステートメント	説明
CLS [1〜3]	テキスト、グラフィック、または両画面を消去します。
LOCATE X,Y	カーソル位置を指定します。
COLOR fg[,bg]	文字色と背景色を指定します。
WIDTH 40 / WIDTH 80	文字幅とグラフィック解像度（320×200 / 640×200）を切り替えます。
BEEP	PSGでMSX1互換のビープ音を鳴らします。
PLAY mml$[,mml$[,mml$]]	AY-3-8910風の最大3チャンネルMMLを演奏します。
REM / '	コメントを記述します。
SPEED n	1秒あたりの実行文数とグラフィック描画ドット数を指定します。

8.15 グラフィック

ステートメント	説明
PSET STEP[,色]	1ドットを描きます。
PRESET STEP	1ドットを黒で消去します。
LINE (X1,Y1)-(X2,Y2)[,色]	線を描きます。始点を省略すると直前の座標を使います。
LINE ...[,色],B	長方形の枠を描きます。
LINE ...[,色またはタイル文字列],BF	単色または8×nタイルで塗りつぶした長方形を描きます。
CIRCLE (X,Y),半径[,色[,開始角[,終了角[,縦横比]]]]	円、楕円、円弧を描きます。角度はラジアンです。
PAINT (X,Y)[,塗色またはタイル文字列[,境界色...]]	閉領域を単色または8×nタイルで塗りつぶします。

9. 関数リファレンス

9.1 数値関数

関数	説明
ABS(x)	絶対値
INT(x)	x以下の最大整数。負数では負方向へ丸めます。
FIX(x)	小数部を切り捨て、0方向へ丸めます。
SGN(x)	正なら1、0なら0、負なら-1
SQR(x)	平方根。負数はエラーです。
SIN(x)	正弦。単位はラジアンです。
COS(x)	余弦。単位はラジアンです。
TAN(x)	正接。単位はラジアンです。
ATN(x)	逆正接
LOG(x)	自然対数。xは正数でなければなりません。
EXP(x)	eのx乗
CINT(x)	16ビット整数へ丸めます。
CSNG(x)	数値として返します。
CDBL(x)	数値として返します。

9.2 乱数

関数	説明
RND / RND(x)	0以上1未満の乱数を返します。
RND(0)	直前と同じ乱数を返します。
RND(負数)	負数を種として乱数系列を設定し、乱数を返します。

9.3 文字列関数

関数	説明
LEN(s$)	文字数。濁点・半濁点は独立した1文字として数えます。
ASC(s$)	先頭文字の1バイト文字コード。文字ROM外ではUnicodeコード値。
VAL(s$)	先頭から数値として読める部分を数値化
CHR$(n)	0〜255はPC-8001/8801系1バイト文字、256以上はUnicodeコード値nの文字。
STR$(x)	数値を文字列化。正数には先頭空白が付きます。
LEFT$(s$,n)	左からn文字
RIGHT$(s$,n)	右からn文字
MID$(s$,開始[,長さ])	指定位置から部分文字列。開始位置は1から数えます。
INSTR([開始,]s$,検索$)	検索文字列の位置。見つからなければ0です。
STRING$(n,文字またはコード)	同じ文字をn個並べます。
SPACE$(n)	空白をn個返します。
HEX$(n)	16進文字列
HEXCHR$(s$)	空白を無視し、16進2桁ごとにコード値0〜255の1文字へ変換します。
OCT$(n)	8進文字列

9.4 画面・キー・時刻

関数	説明
POS(0)	現在のカーソル列を1起点で返します。
CSRLIN	現在のカーソル行を1起点で返します。
POINT(X,Y)	グラフィック画面の色番号を返します。範囲外は-1です。
INKEY$	押されているキーを待たずに1文字取得。なければ空文字列です。
INPUT$(n)	キー入力を待ち、n文字取得します。
TIME$	現在時刻をHH:mm:ss形式で返します。
DATE$	現在日付をMM-dd-yyyy形式で返します。

GUI版の矢印キーはINKEY$で次のコードになります。

キー	値
右	CHR$(28)
左	CHR$(29)
上	CHR$(30)
下	CHR$(31)

9.5 仮想メモリ・互換関数

関数	説明
PEEK(アドレス)	仮想メモリの1バイトを読み出します。
FRE(x)	互換用の空きメモリ値を返します。

10. ファイルの保存と読み込み

10.1 保存フォルダ

GUI版の標準保存先は次のフォルダです。

```
~/Documents/RetroBASIC/
```

フォルダが存在しない場合、最初のSAVEで自動作成されます。拡張子を省略すると.basが 自動的に付きます。

```
SAVE "GAME"
```

この例では~/Documents/RetroBASIC/GAME.basへ保存します。

環境変数RETROBASIC_DIRを設定して起動すると、別の保存フォルダを指定できます。

10.2 読み込み

```
LOAD "GAME"
```

現在のプログラムを置き換えます。読み込み後はRUNで実行します。

読み込みと実行を一度に行うこともできます。

```
RUN "GAME"
```

10.3 プログラムの統合

```
MERGE "ROUTINES"
```

現在のプログラムを残してファイルを統合します。同じ行番号があれば、読み込んだ行で 置き換えます。

10.4 一覧と削除

```
FILES
KILL "OLDGAME"
```

KILLで削除したファイルは元に戻せないため、ファイル名を確認してから実行してください。

10.5 BASICファイル形式

保存ファイルはUTF-8のプレーンテキストです。通常のテキストエディタでも閲覧・編集できます。

```
10 PRINT "HELLO"
20 END
```

11. 実行速度

RetroBASICは、既定で1秒あたり4000文に実行速度を調整します。これにより、当時よく使われた 空のFORループによる待ち時間が、現代のMacでも意味を持ちます。 LIST のスクロール速度もこの値に連動します。既定の SPEED 4000 では当時のマシンに近い 約30画面行/秒で表示され、値を小さくすると遅く、大きくすると速くなります。 PSET、PRESET、LINE、CIRCLE、PAINT の描画速度にも連動し、既定値では 約4000ドット/秒です。線や円が伸び、塗りつぶしが水平走査線ごとに広がる途中経過を表示します。

```
FOR W=1 TO 300:NEXT W
```

SPEEDはRetroBASIC独自の拡張命令です。

```
SPEED 0
```

0を指定するとプログラム実行、LIST 表示、グラフィック描画の速度制限を解除します。

```
SPEED 4000
```

既定速度へ戻します。指定範囲は0〜100,000,000で、文/秒および概ね描画ドット/秒を表します。

12. エラーと対処

プログラム実行中のエラーには行番号が付きます。

```
Division by zero in 120
```

メッセージ	主な原因と対処
Syntax error	命令、括弧、区切り、引用符などの書式を確認します。
Type mismatch	数値と文字列を混在させていないか確認します。
Illegal function call	関数の引数、WIDTH、SPEED、POKEなどの値を確認します。
Illegal direct	RENUMやAUTOをプログラム内ではなくイミディエイトモードで実行します。
Overflow	数値または整数変数の範囲を超えています。
Division by zero	/、\、MODの除数が0です。
Undefined line number	GOTO、GOSUB、RUNの行番号が存在しません。
Subscript out of range	配列の添字または次元数を確認します。
Duplicate Definition	同じ配列を再度DIMしています。ERASEしてから宣言します。
NEXT without FOR	対応するFORがありません。
WEND without WHILE	対応するWHILEがありません。
RETURN without GOSUB	対応するGOSUBがありません。
Out of DATA	READする項目数に対してDATAが不足しています。
File not found	ファイル名と保存フォルダを確認します。
Device I/O error	保存先のアクセス権や空き容量を確認します。
Can't continue	CONTできる停止位置が残っていません。RUNし直します。
?Redo from start	INPUTの形式または項目数が合いません。入力し直します。

12.1 プログラムを止める

実行中にControl+CまたはCommand+.を押します。

```
Break in 100
Ok
```

停止位置から再開できる場合はCONTを使います。最初からやり直す場合はRUNを使います。

12.2 反応がないように見える場合

- INPUTやINPUT$でキー入力を待っていないか確認します。
- 無限ループになっている場合はControl+CまたはCommand+.で中断します。
- SPEEDが非常に小さい値になっていないか確認します。

13. サンプルプログラム

samplesフォルダに次のプログラムがあります。

ファイル	内容	主な命令
HELLO.bas	色付き文字と文字列パターン	COLOR、TAB、STRING$、TIME$
GUESS.bas	1〜100の数当てゲーム	INPUT、RND、IF
STARS.bas	ランダムな星空表示	LOCATE、COLOR、RND
CATCH.bas	落下物を受け止めるゲーム	INKEY$、LOCATE、矢印キー
GRAPHICS.bas	PC-8801風の図形表示	WIDTH、LINE、CIRCLE、PAINT
TILE_PAINT.bas	X1 Hu-BASIC互換タイルの表示	HEXCHR$、LINE BF、PAINT
3D_MAZE_SAMPLE.bas	ランダム生成される3D迷路	WIDTH、INKEY$、LINE、BEEP
MYSTERY_ROOM.bas	1部屋から脱出するコマンド式アドベンチャー	LINE INPUT、文字列関数、タイルPAINT
MUSIC.bas	3チャンネルのPSG音楽	PLAY、MML、トーン、ノイズ

GUI版でファイル内容をコピーして貼り付け、RUNすることもできます。CLI版ではファイル名を 直接指定できます。

```
./retrobasic-cli samples/HELLO.bas
```

14. 制限事項

次の機能には対応していません。

- SCREEN、640×400モノクロ、VRAMページ、WINDOW、GET/PUTなどの高度なグラフィック機能
- EDIT
- 特定機種のROM、VRAM、I/Oポートなどの実ハードウェア再現
- 当時の単精度浮動小数点特有の丸め誤差

POKEとPEEKはRetroBASIC内部の64KB仮想メモリに対して動作します。

CLI版のINKEY$は常に空文字列を返すため、CATCH.basのようなリアルタイムキー入力を 使うプログラムはGUI版で実行してください。

RetroBASICは、古いBASICのプログラムを完全互換で動かすことよりも、行番号、青や黒の テキスト画面、直接命令、スクリーン編集、素朴なゲーム作りといった当時の体験を楽しむことを 目的としています。